

Machine Learning Project

머신러닝 프로젝트



세계 최대 게임플랫폼 스팀(STEAM)의 세일 할인가격 예측

STEAM SALE PRICE PREDICTION

Index 목차



머신러닝 프로젝트 스팀 할인 가격 예측

STEAM SALE PRICE PREDICTION

01. 주제선정 및 데이터 수집

- 프로젝트 소개
- 스크롤링
- 셀레니엄

02. EDA / 전처리

- 탐색적 데이터 분석(Exploratory Data Analysis)
- 결측값 처리, 데이터 타입 변환, 특성(피쳐) 선택
- 이상치 제거, 원-핫 인코딩, 데이터 분리, 정규화(스케일링)

03. 머신러닝 학습 / 예측 / 평가

- 알고리즘 선택
- 알고리즘 학습 및 예측
- 회귀 모델 평가 MAE, MSE, R2 스코어

04. 알고리즘 최적화 및 활용

- 하이퍼파라미터 튜닝 : 그리드 서치(Grid Search)
- 스팀 할인 가격 예측 알고리즘 결론
- 팀원 소개, 질의응답

01. 주제선정 및 데이터 수집



머신러닝 프로젝트 스팀 할인 가격 예측

STEAM SALE PRICE PREDICTION

01. 주제선정 및 데이터 수집

- 프로젝트 소개
- 스크롤링
- 셀레니엄

02. EDA / 전처리

- 탐색적 데이터 분석(Exploratory Data Analysis)
- 결측값 처리, 데이터 타입 변환, 특성(피쳐) 선택
- 이상치 제거, 원-핫 인코딩, 데이터 분리, 정규화(스케일링)

03. 머신러닝 학습 / 예측 / 평가

- 알고리즘 선택
- 알고리즘 학습 및 예측
- 회귀 모델 평가 MAE, MSE, R2 스코어

04. 알고리즘 최적화 및 활용

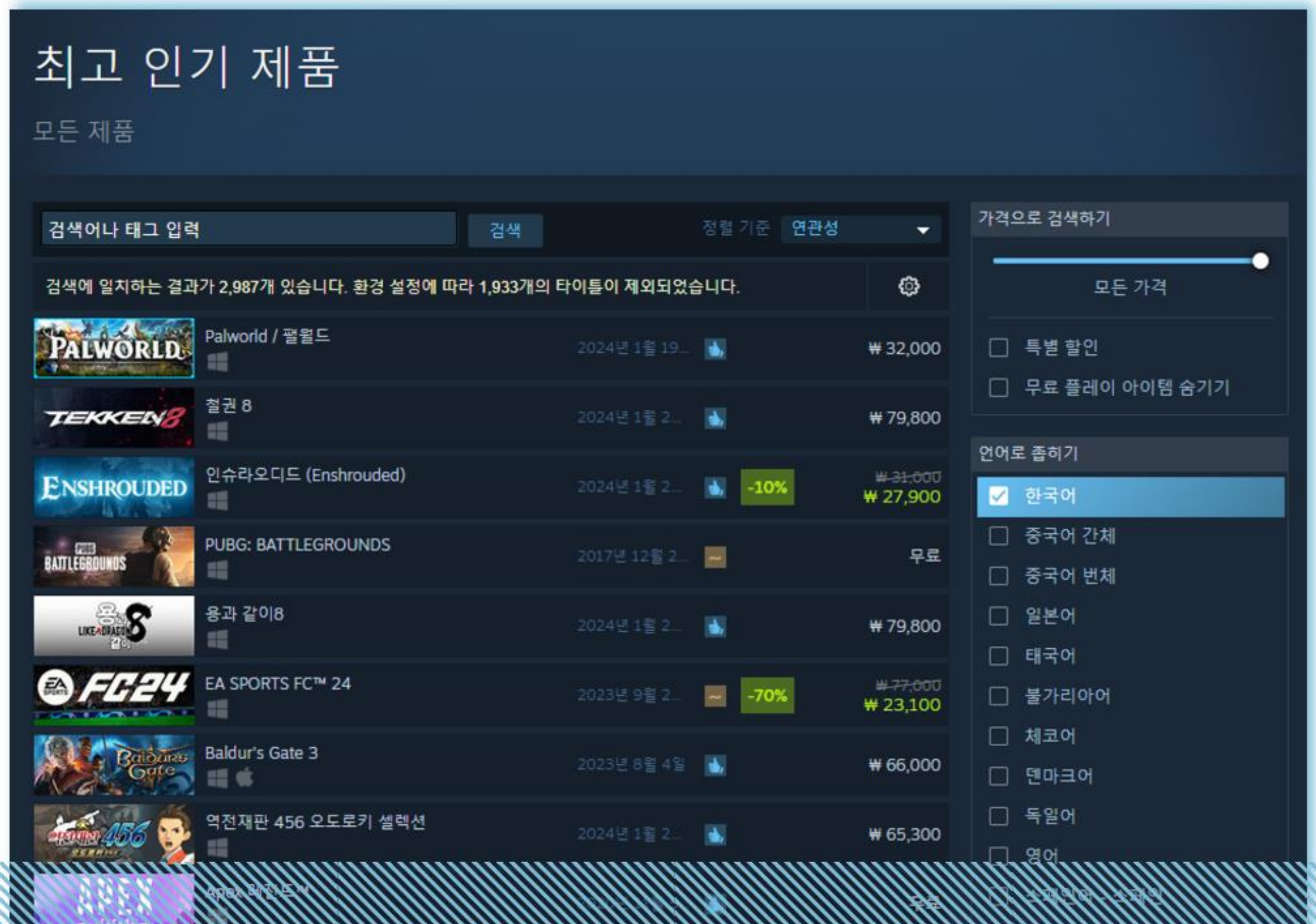
- 하이퍼파라미터 튜닝 : 그리드 서치(Grid Search)
- 스팀 할인 가격 예측 알고리즘 결론
- 팀원 소개, 질의응답



스팀(STEAM)이란?

2003년에 시작된 세계 최대 규모의 전자 소프트웨어 유통 플랫폼
5만여개의 게임 / 10억 개의 계정 / 월 평균 사용자 1억 3천만 명

스팀 이전 — CD 유통 물류비용 / 업데이트, 저장 파일



프로젝트 소개

스팀 세일

전세계의 게임 개발사와 유통 퍼블리셔는
자신들의 게임을 스팀에 출시한 후
시즌별, 각종 이벤트별로 매출향상을 위한 **가격 할인**을 진행합니다.



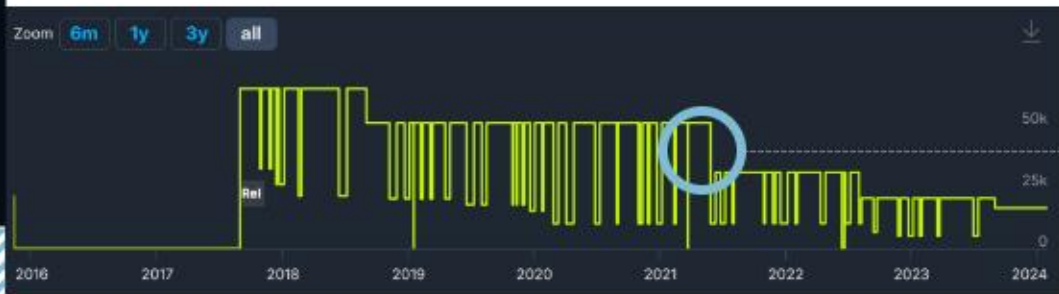
프로젝트 소개

니즈

자신이 구매하기 원하는 게임이
세일 기간에 얼마나 할인하는지는
일반적인 게이머들의 중요 관심사입니다.

할인 정보 사이트

전세계의 각종 스팀 게임 할인정보 사이트에서
필요한 정보를 모아 머신러닝에 필요한 DB를 구성합니다.



Games Data Intelligence Tool			
Search for a game, developer or publisher			
Top Grossing Games			
Released in the last 14 days			
1	Patworld	2024-01-18	\$335M
2	Patworld	2024-01-24	\$23M
3	Patworld	2024-01-25	\$12M
4	Patworld	2024-01-31	\$5M
5	Like a Dragon: Infinite Wealth	2024-01-25	\$4M
6	UNDER NIGHT IN BIRTH II Sys:Celes	2024-01-24	\$48.99

3소스

store.steampowered.com

SteamDB.info

GameSensor.info

필요 데이터

1. 게임정보 데이터

2. 세일정보 히스토리 데이터

3. 게임 판매수익 데이터

```
1 import requests
2 url = 'https://store.steampowered.com/'
3 res = requests.get(url)
4 res
```

<Response [200]>

[403]인 경우에도 개발자 도구에서 'Referer', 'User_Agent' 등을 headers에 포함할 경우 [200]이 될 수 있음.

```
res = requests.get(url, headers=headers)
```

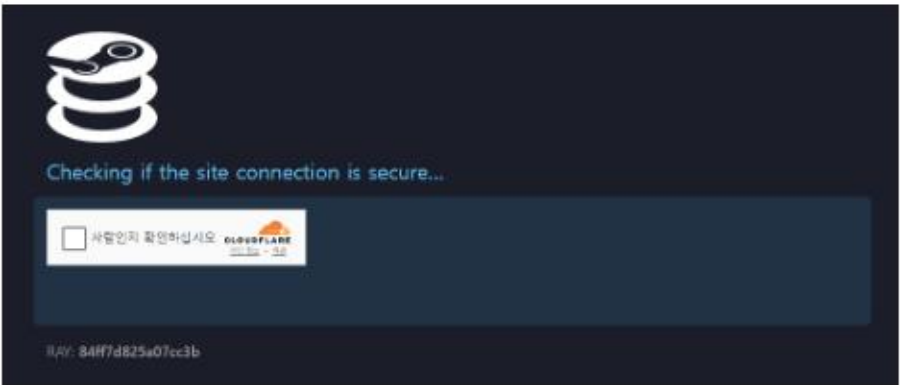
머신러닝 DB구성

크롤링

셀레니엄

셀레니엄 차단

```
driver = webdriver.Chrome()
```





3개의 사이트에서
각각 자료를 수집하고 공유하기 위해
매 단계의 결과물을
파일 형태로 저장했습니다.

크롤링을 위한 소스도
DFMaker.py 파이썬 파일로 저장하여
다른 팀원이 해당 작업을 수행할 경우
함수로 호출하여 실행하였습니다.

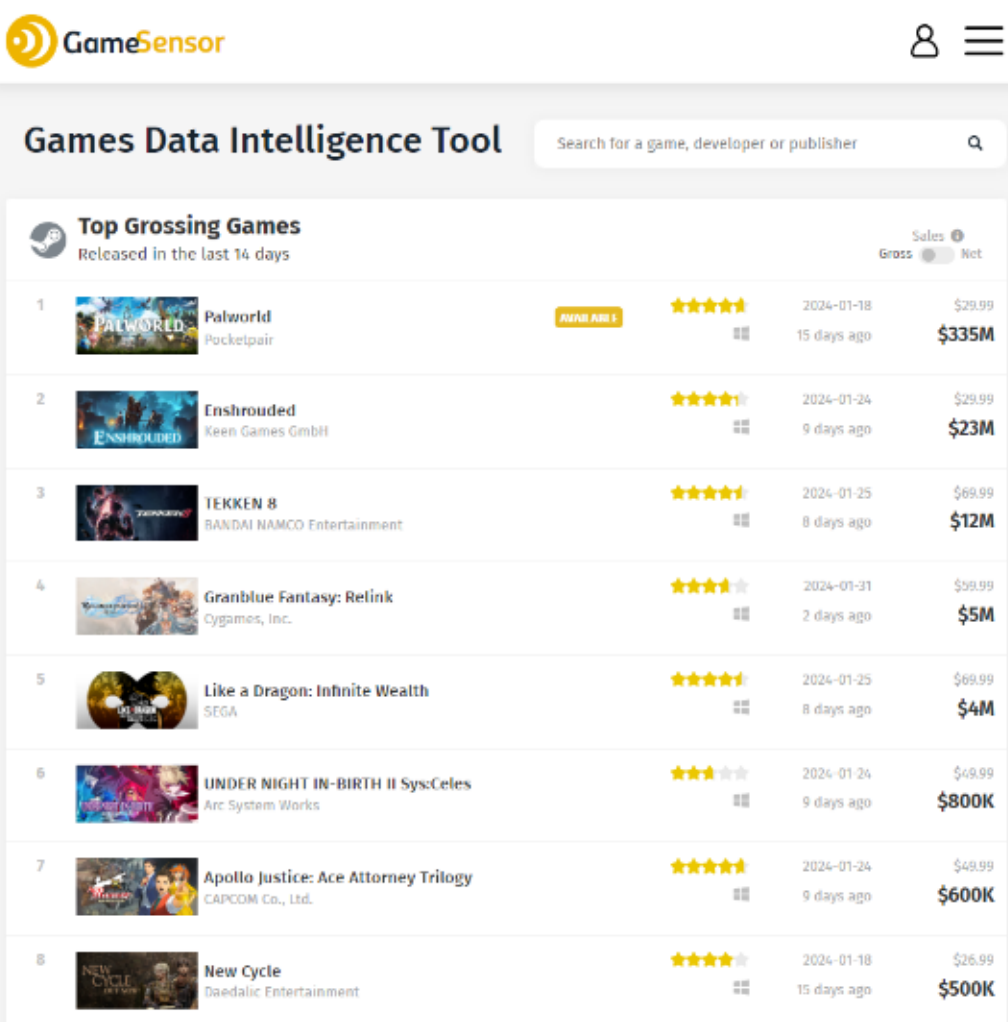
crawling

크롤링

```
1 import os
2 import re
3 import pandas as pd
4
5 import DFMaker
6
7 # 데이터 프레임 구성하기
8 game_info_columns = [
9     'AppID',
10    'Name',
11    'Developer',
12    'Company',
13    'price',
14    'latest rating',
15    'latest rating counter',
16    'all rating',
17    'all rating counter',
18    'release date',
19    'category',
20    'DLC_Game']
21
22 csv_root_path = 'game_data'
23 csv_folder_list = os.listdir(csv_root_path)
24
25 # csv 파일 불러와서 체크하기
26 try:
27     game_df = pd.read_csv(r'static_data\game_info.csv')
28 except:
29     game_df = pd.DataFrame(columns=game_info_columns)
30
31 try:
32     game_df = game_df.drop(columns=['Unnamed: 0.1', 'Unnamed: 0'])
33 except:
34     pass
35
36 file_list = []
37
38 # 존재하는지 비교
39 for item in csv_folder_list:
40     reg = re.compile('\d+')
41     file = reg.findall(item)
42
43     if (game_df['AppID'] == int(file[0])).sum() > 0:
44         continue
45
46     file_list.append(file[0])
47
48 new_df = pd.DataFrame(columns=game_info_columns)
49
50 error_list = []
51
52 # 게임 추가해서 더하기
53 for item in file_list:
54     try:
55         game_info_df = DFMaker.create_game_info(item)
56         new_df = pd.concat([new_df, game_info_df])
57     except:
58         print('{} error'.format(item))
59         error_list.append(item)
60
61 game_df = pd.concat([game_df, new_df])
62 game_df.to_csv('static_data\game_info.csv')
```

```
7 def create_game_info(game_id):
8
9     # 데이터 프레임 구성하기
10    game_info_columns = [
11        'AppID',
12        'Name',
13        'Developer',
14        'Company',
15        'price',
16        'latest rating',
17        'latest rating counter',
18        'all rating',
19        'all rating counter',
20        'release date',
21        'category',
22        'DLC_Game']
23
24    df = pd.DataFrame(columns=game_info_columns)
25
26    # 게임 인덱스 구성하기
27    print(game_id)
28    url = 'https://store.steampowered.com/app/{}/'.format(game_id)
29    res = requests.get(url)
30    soup = BeautifulSoup(res.text)
31
32    # 'AppID'
33    rev_game_id = game_id
34    # 'Name', 게임 이름
35    rev_game_name = (soup.select('#appHubAppName')[0].text)
36    # 'Developer', 개발자
37    rev_game_dev = (soup.select('#game_highlights > div.rightcol > div > div.glance_ctn_responsive_left > div:nth-child(3) > div.summary, column > a')[0].text)
38    # 'Company', 배급사
39    company_el = soup.select('#game_highlights > div.rightcol > div > div.glance_ctn_responsive_left > div:nth-child(4) > div.summary, column > a')
40    rev_game_company = []
41    for company in company_el:
42        rev_game_company.append(company.text)
43
44    # 'price', 현재가격
45    try:
46        rev_game_price = (soup.select('.game_area_purchase_game > div.game_purchase_action > div > div.game_purchase_price.price')[0].text.strip())
47    except:
48        try:
49            rev_game_price = (soup.select('.game_area_purchase_game > div.game_purchase_action > div > div.discount_block, game_purchase_discount > div.discount_prices > div.discount_original_price')[0].text.strip())
50        except:
51            rev_game_price = 0
52
53    print('게임가격 : ', rev_game_price)
54    # 'latest rating', 최근 평가
55    try:
56        rev_game_late_rating = (soup.select('#review_histogram_rollup_section > div.user_reviews_summary_bar > div > span, game_review_summary')[0].text)
57    except:
58        rev_game_late_rating = np.nan
59    # 'latest rating counter', 최근 평가 인원
60    try:
61        rev_game_late_rating_counter = (soup.select('#review_histogram_recent_section > div.user_reviews_summary_bar > div > span:nth-child(3)')[0].text)
62    except:
63        rev_game_late_rating_counter = 0
64    # 'all rating', 모든 평가
65    try:
66        #appReviewsAll_responsive > div.summary.column > span.game_review_summary.mixed
67        rev_game_all_rating = (soup.select('#review_histogram_rollup_section > div.user_reviews_summary_bar > div > span, game_review_summary')[0].text)
68    except:
69        rev_game_all_rating = np.nan
```

같은 항목이어도 데이터가 다양한 형태로 들어가 있기 때문에
결과값을 보고 예외 형태의 path를 조정해주면서
try / except 로 모든 항목의 예외값을 조정해 주었습니다.



셀레니엄을 사용할 때에도
기존에 저장된 파일을 불러와
기존에 데이터가 있는 자료만
수집하도록 하여 시간을 절약하였습니다.

selenium
셀레니엄

```
1 import pandas as pd
2 from selenium import webdriver
3 from selenium.webdriver.common.by import By
4 from selenium.webdriver.common.keys import Keys
5 from bs4 import BeautifulSoup
6
7 driver = webdriver.Chrome()
8 account_columns = ['AppID', 'Total_Gross', 'Total_Net', 'Total_owners']
9 game_info_df = pd.read_csv('static_data\game_info.csv') # 게임 정보 가져오기
10 game_info_df['AppID'].sort_values()
11 try:
12     account_df = pd.read_csv('static_data\game_account_info.csv') # Account 정보 가져오기
13 except:
14     account_df = pd.DataFrame(columns=account_columns)
15
16 add_file_list = [] # 작업 완료된 데이터 필터링
17 error_file_list = []
18 for game_id in game_info_df['AppID'].sort_values().values:
19     if (account_df['AppID'] == game_id).sum() > 0:
20         continue
21     print('작업 게임 ID : ', game_id)
22     try:
23         url = 'https://gamesensor.info/app/{}'.format(game_id)
24         driver.get(url)
25         html = driver.page_source
26         soup = BeautifulSoup(html, 'html.parser')
27
28         total_gross = '' # 데이터 파싱
29         total_net = ''
30         total_owners = ''
31         for item in soup.select('#user_page > div.col-lg-6'):
32             if str(item.select('div.text')[0]).find('Total Gross') != -1:
33                 recv = item.select('div > div.value')[0].text.strip()
34                 total_gross = recv.split('$')[1]
35                 total_net = recv.split('$')[2]
36             if str(item.select('div.text')[0]).find('Total Owners') != -1:
37                 total_owners = item.select('div > div.value')[0].text.strip()
38
39         result_total_gross = 0
40         result_total_net = 0
41         result_total_owners = 0
42         total_gross = total_gross.replace('>', '')
43         total_gross = total_gross.replace('$', '')
44         total_net = total_net.replace('>', '')
45         total_net = total_net.replace('$', '')
46         total_owners = total_owners.replace('>', '')
47         total_owners = total_owners.replace('$', '')
48         if 'M' in total_gross:
49             result_total_gross = float(total_gross.replace('M', '')) * 1000000
50         if 'K' in total_gross:
51             result_total_gross = float(total_gross.replace('K', '')) * 1000
52         if 'M' in total_net:
53             result_total_net = float(total_net.replace('M', '')) * 1000000
54         if 'K' in total_net:
55             result_total_net = float(total_net.replace('K', '')) * 1000
56         if 'M' in total_owners:
57             result_total_owners = float(total_owners.replace('M', '')) * 1000000
58         if 'K' in total_owners:
59             result_total_owners = float(total_owners.replace('K', '')) * 1000
60         new_account_df = pd.DataFrame(columns=account_columns) # 데이터 추가하기
61         new_account_df.loc[0] = [game_id, result_total_gross, result_total_net, result_total_owners]
62         account_df = pd.concat([account_df, new_account_df])
63         account_df.to_csv('static_data\game_account_info.csv')
64     except:
65         error_file_list.append(game_id) # error 파일 검증
66         print('Error : ', game_id)
67
68 account_df.to_csv('static_data\game_account_info.csv') # 저장
```

모든 판매금액이 천단위를 K,
백만위를 M으로 표시하고 있기 때문에
해당 알파벳을 숫자로 바꿔주는 조건을 추가합니다.

다른 사이트에서 수집한 파일을 불러와
셀레니엄으로 수집한 파일과 merge하여
모든 정보를 통합해 파일로 저장합니다.

```
1 account_df = pd.read_csv('static_data\game_account_info.csv')
2 account_df = account_df.drop(['Unnamed: 0.1', 'Unnamed: 0'], axis=1)
3 account_df
4
5 game_info_df = pd.read_csv('static_data\game_info.csv')
6 game_info_df = game_info_df.drop(['Unnamed: 0', 'Unnamed: 0.2'], axis=1)
7 game_info_df
8
9 df = pd.merge(game_info_df, account_df, on='AppID', how='left')
10 df.to_csv('game_account_merge.csv')
```

02. EDA/ 전처리



머신러닝 프로젝트 스팀 할인 가격 예측

STEAM SALE PRICE PREDICTION

01. 주제선정 및 데이터 수집

- 프로젝트 소개
- 스크롤링
- 셀레니엄

02. EDA / 전처리

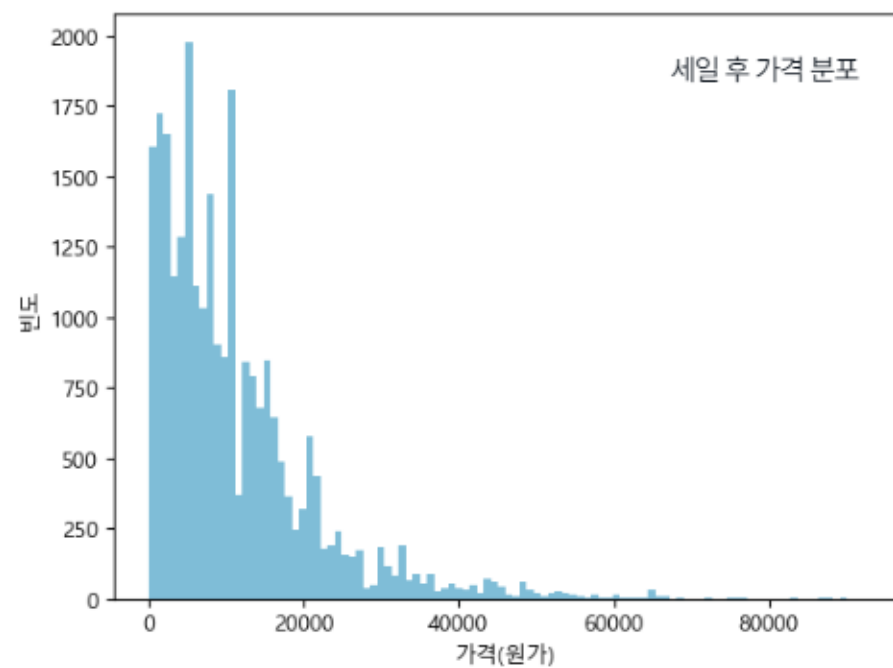
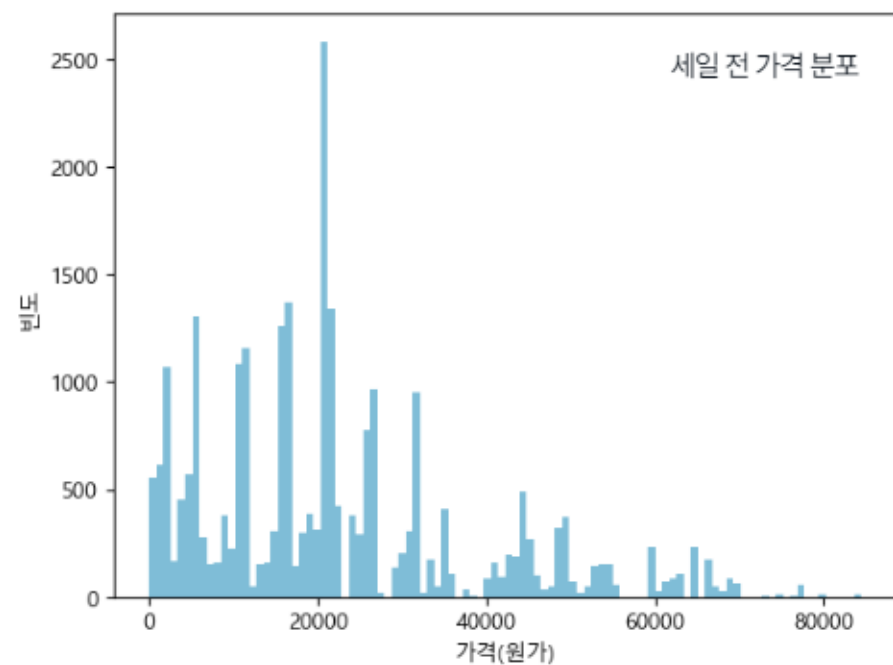
- 탐색적 데이터 분석(Exploratory Data Analysis)
- 결측값 처리, 데이터 타입 변환, 특성(피쳐) 선택
- 이상치 제거, 원-핫 인코딩, 데이터 분리, 정규화(스케일링)

03. 머신러닝 학습 / 예측 / 평가

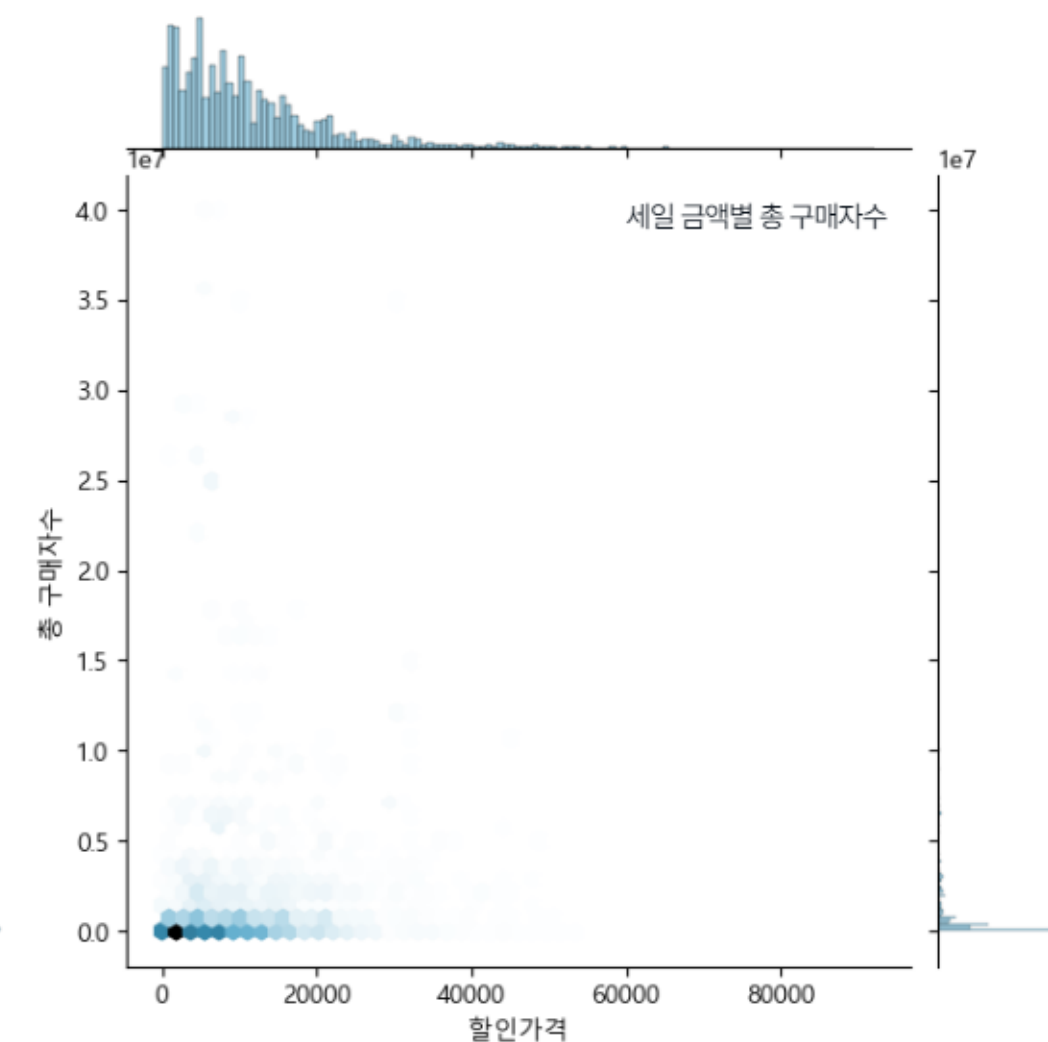
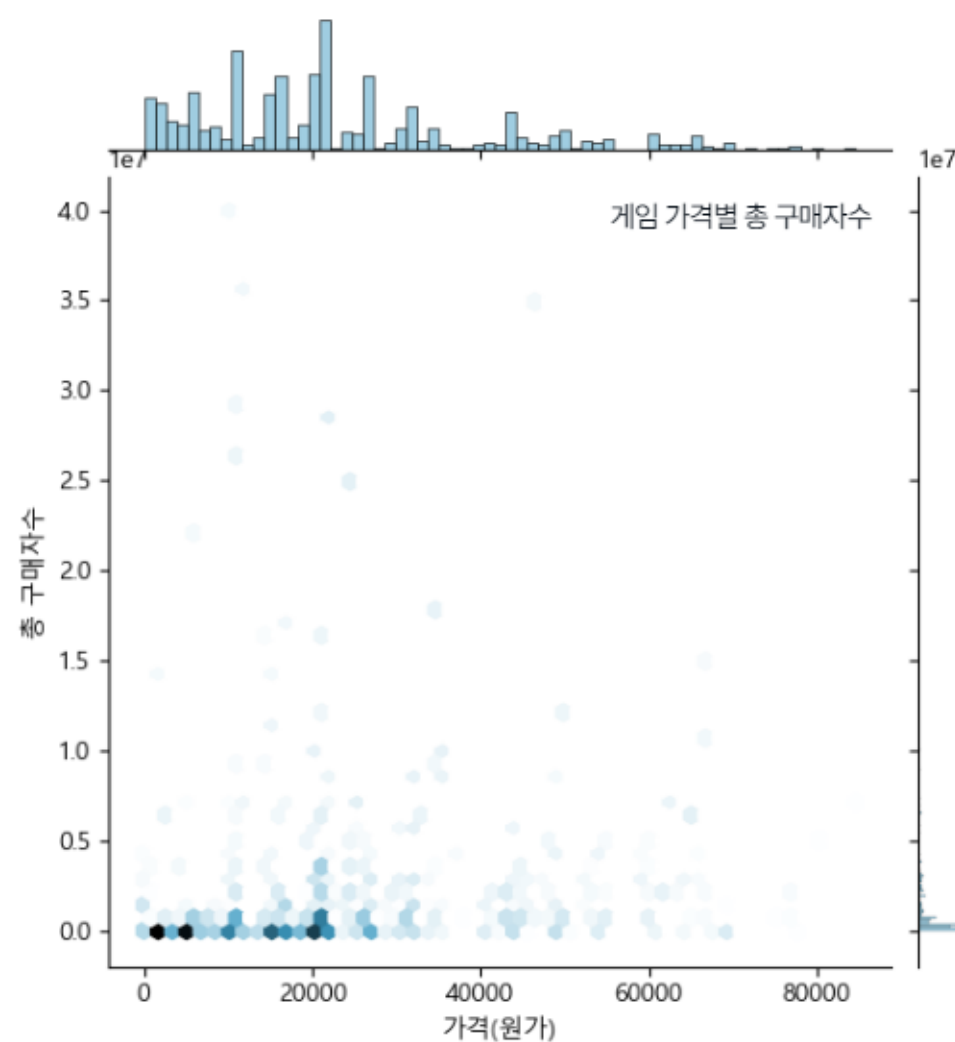
- 알고리즘 선택
- 알고리즘 학습 및 예측
- 회귀 모델 평가 MAE, MSE, R2 스코어

04. 알고리즘 최적화 및 활용

- 하이퍼파라미터 튜닝 : 그리드 서치(Grid Search)
- 스팀 할인 가격 예측 알고리즘 결론
- 팀원 소개, 질의응답



```
merged_df['price'].plot(kind='hist', bins=100, color='#7fbdd7')
```



```
sns.jointplot(x='Final price', y='Total_Net', data=merged_df, color='#7fbdd7', kind='hex')
```

Exploratory Data Analysis

EDA : 탐색적 데이터 분석

EDA란?

EDA는 데이터를 분석하고 처리하기 전에 그래프나 통계적인 방법으로 데이터를 탐색하고 자료를 직관적으로 바라보며 통찰을 얻는 과정입니다.

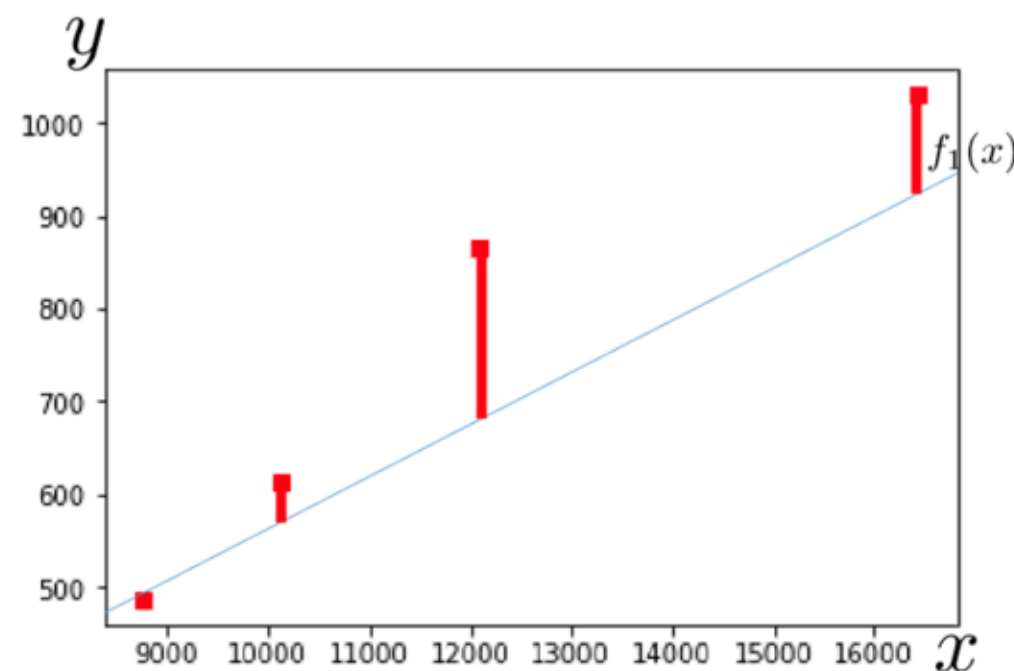
머신러닝 요약

회귀모델 Regression

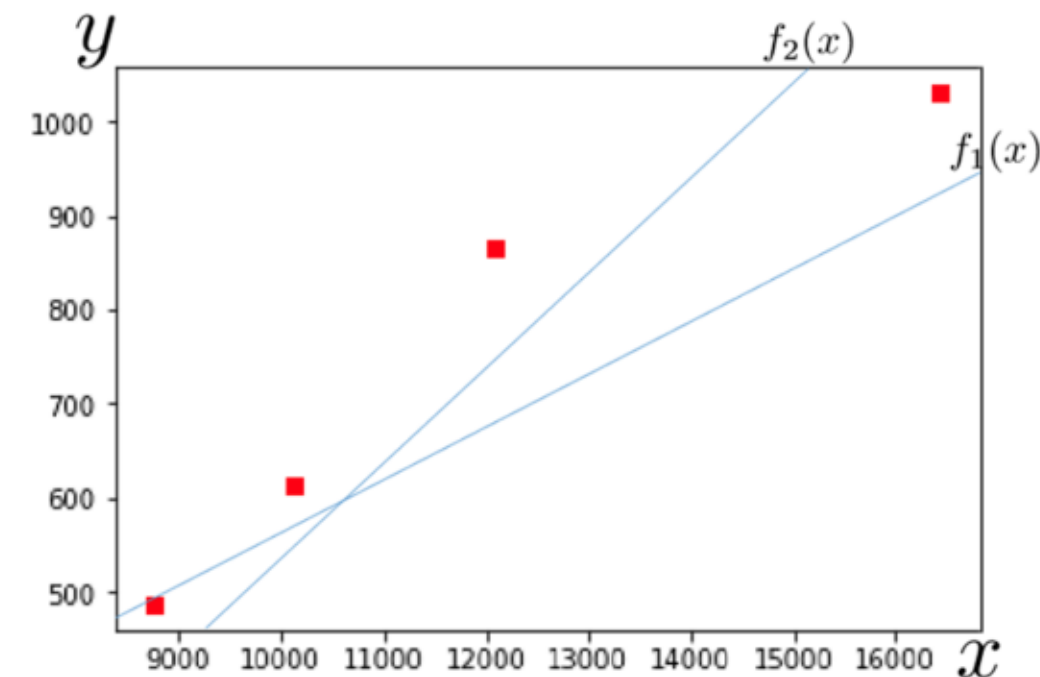
- X 데이터를 넣어 연속된 숫자 Y를 예측하기
- **가격**, 주가, 판매량 등

분류모델 Classification

- X 데이터를 넣어 불연속 값 Y를 분류하기
- Y/N, 질병 분류, 카테고리 그룹화



오차가 작을수록 좋다



모든 점과의 거리가 가장 작은 선을 찾을때까지 계산

전처리 목표

알고리즘이 **오차**를 수학적으로 계산할 수 있도록
모든 데이터를 불필요하거나 비어있는 값이 없도록 정리하고,
int(숫자)나 object(오브젝트) 자료형이 되도록 변환합니다.

게임정보와 판매수익을 합친 데이터 프레임

	Unnamed: 0	AppID	Name	Developer	Company	price	latest rating	latest rating counter	all rating	all rating counter	release date	category	DLC_Game	Total_Gross	Total_Net	Total_owners
0	0	10	Counter-Strike	Valve	['Valve']	10500	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	1 Nov, 2000	['Action']	NaN	50000000.0	30000000.0	9300000.0
1	1	1001590	Grab Lab	Digital Melody	['Forever Entertainment S. A.']	5500	Mostly Positive	0	Mostly Positive	13	21 Jun, 2019	['Action', 'Casual', 'Indie']	NaN	5000.0	5000.0	10000.0
2	2	1001880	aMAZE Valentine	Blender Games	['Blender Games']	2300	Positive	0	Positive	38	28 Dec, 2018	['Adventure', 'Casual', 'Indie', 'Simulation', ...]	NaN	5000.0	5000.0	10000.0
3	3	1003590	Tetris® Effect: Connected	Monstars Inc.	['Enhance']	41000	Overwhelmingly Positive	114	Overwhelmingly Positive	5938	17 Aug, 2021	['Casual', 'Indie']	NaN	3000000.0	2000000.0	100000.0
4	4	1004330	My Exercise	New Deer	['Playables', 'New Deer']	3400	Very Positive	0	Very Positive	382	26 Aug, 2020	['Action', 'Casual', 'Indie', 'Racing', 'Sports']	NaN	30000.0	20000.0	10000.0

불필요한 컬럼 삭제

세일정보 데이터 프레임

Unnamed: 0	DateTime	Final price	Sale Start	Sale End	Origin Price	Sale Rate	Ready to Sale	Sale Period	Game_ID	Start Year	Start month	Start day	End Year	End month	End day
0	2015-11-25 18:24:05	5250.0	2015-11-25 18:24:05	2015-12-01 18:39:34	10500.0	0.50	21 days 00:54:41	6 days 00:15:29	10	2015	11	25	2015	12	1
1	2015-12-22 19:34:15	5250.0	2015-12-22 19:34:15	2016-01-04 19:21:20	10500.0	0.50	32 days 00:10:02	12 days 23:47:05	10	2015	12	22	2016	1	4
2	2016-02-05 19:31:22	2620.0	2016-02-05 19:31:22	2016-02-12 18:47:14	10500.0	0.75	285 days 01:47:07	6 days 23:15:52	10	2016	2	5	2016	2	12
3	2016-11-23 20:34:21	2620.0	2016-11-23 20:34:21	2016-12-01 16:14:00	10500.0	0.75	21 days 01:44:51	7 days 19:39:39	10	2016	11	23	2016	12	1
4	2016-12-22 17:58:51	2620.0	2016-12-22 17:58:51	2017-01-02 18:23:32	10500.0	0.75	171 days 02:48:31	11 days 00:24:41	10	2016	12	22	2017	1	2

날짜 타입의 자료를 기간 등 다른 형태로 변환하여 컬럼 생성 후 삭제

하나의 파일로 병합

merged_df = pd.merge(all_sale_df, game_info_df, left_on='Game_ID', right_on='AppID', how='right')

```
1 # 출시일 전처리
2 game_info_df['release date'] = game_info_df['release date'].str.replace(',','') # 쉼표(,)가 있는 값이 있음 예)일,월,년
3 temp_index = game_info_df[game_info_df['release date'].apply(lambda x : len(x.split()) < 3)].index # 일 값이 없는 경우가 있음
4 game_info_df['release date'].iloc[temp_index] = '1 ' + game_info_df['release date'].iloc[temp_index] # 일 값에 1 추가
5
6 # 출시일 컬럼 생성
7 game_info_df['last Day'] = (pd.to_datetime(datetime.today()) - pd.to_datetime(game_info_df['release date'])).apply(lambda x : str(x).split()[0])
8
9 # 기존 날짜 컬럼 삭제
10 game_info_df = game_info_df.drop(columns=['release date'])
```

Python

결측값 처리

NaN 값

값이 비어있는 데이터가 없도록 처리합니다.
NaN값이 있으면 특정 값이나 평균값 등으로 채워넣거나
해당 행이나 컬럼을 삭제하기도 합니다.

```
1 # Nan 값처리
2 merged_df['price'] = merged_df['price'].str.extract('(\d+)') # 원 -> 숫자만 추출
3 merged_df['price'] = merged_df['price'].fillna(0)
4
5 merged_df['Ready to Sale Day'] = merged_df['Ready to Sale Day'].fillna(0)
6 merged_df['Sale Peroid Day'] = merged_df['Sale Peroid Day'].fillna(0)
7
8 merged_df = merged_df.dropna() # NaN값이 있는 행 삭제
```

✓ 0.0s

Python

데이터 타입 변환 astype()

각 컬럼의 자료형이 int(숫자)나 object(오브젝트)가 되도록 변환합니다.

```

1 # 타입 형 형변환
2 merged_df['Unnamed: 0'] = merged_df['Unnamed: 0'].astype(int) + 1
3
4 merged_df['Start Year'] = merged_df['Start Year'].astype('int16')
5 merged_df['Start month'] = merged_df['Start month'].astype('int16')
6 merged_df['Start day'] = merged_df['Start day'].astype('int16')
7
8 merged_df['End Year'] = merged_df['End Year'].astype('int16')
9 merged_df['End month'] = merged_df['End month'].astype('int16')
10 merged_df['End day'] = merged_df['End day'].astype('int16')
11
12 merged_df['Start Year'] = merged_df['Start Year'].astype(object)
13 merged_df['Start month'] = merged_df['Start month'].astype(object)
14 merged_df['Start day'] = merged_df['Start day'].astype(object)
15
16 merged_df['End Year'] = merged_df['End Year'].astype(object)
17 merged_df['End month'] = merged_df['End month'].astype(object)
18 merged_df['End day'] = merged_df['End day'].astype(object)
19 merged_df['Sale Start'] = pd.to_datetime(merged_df['Sale Start'])
20
21 merged_df['price'] = merged_df['price'].astype('int32')
22 merged_df['last Day'] = merged_df['last Day'].astype('int32')
23
24 merged_df['Game_ID'] = merged_df['Game_ID'].astype(int)
25 merged_df['Final price'] = merged_df['Final price'].astype('int32')
26 merged_df['Sale Peroid Day'] = merged_df['Sale Peroid Day'].astype('int32')
27 merged_df['Ready to Sale Day'] = merged_df['Ready to Sale Day'].astype('int32')
28
29 merged_df['latest rating counter'] = merged_df['latest rating counter'].astype('int32')
30 merged_df['all rating counter'] = merged_df['all rating counter'].astype('int32')

```

✓ 0.0s
Python

특성(피쳐) 선택 drop()

불필요한 컬럼을 미리 삭제하면 머신러닝 속도를 향상시킬 수 있습니다.

```

1 # 데이터 분리 후 필요없는 컬럼 삭제
2 # 세일 간격(날짜데이터), 게임 원가(게임정보에 있음), 할인율(불필요), 세일 기간(날짜데이터),세일 종료일(날짜데이터) 출시일(날짜데이터)
3 all_sale_df = all_sale_df.drop(columns=['Ready to Sale','Origin Price','Sale Rate','Sale Period','Sale End','DateTime'])

```

✓ 0.0s
Python

컬럼 순서 정렬 및 한글화

```

1 new_order = ['Game_ID', 'Unnamed: 0', 'last Day', 'Start Year', 'Start month','Start day', 'End Year', 'End month', 'End day', 'Sale Peroid Day', 'Ready to Sale Day', 'Developer', 'Company', 'latest rating', 'latest rating counter', 'all rating', 'all rating counter', 'category','Total_Gross', 'Total_Net', 'Total_owners', 'price', 'Final price']
2 new_order_df = merged_df.reindex(columns=new_order)

```

✓ 0.0s
Python

```

1 new_order_df.rename(columns={
2     'Game_ID': '게임 ID',
3     'Unnamed: 0': '게임별 세일 순서',
4     'Start Year': '세일시작 연도',
5     'Start month': '세일시작 월',
6     'Start day': '세일시작 일',
7     'End Year': '세일종료 연도',
8     'End month': '세일종료 월',
9     'End day': '세일종료 일',
10    'Sale Peroid Day': '세일 기간',
11    'Ready to Sale Day': '세일 간격',
12    'last Day': '출시일수',
13    'Developer': '제작 스튜디오',
14    'Company': '배급사',
15    'latest rating': '최근 평가',
16    'latest rating counter': '최근 평가수',
17    'all rating': '종합 평가',
18    'all rating counter': '종합 평가수',
19    'category': '게임 장르',
20    'Total_Gross': '총 판매금액',
21    'Total_Net': '총 순이익',
22    'Total_owners': '총 구매자수',
23    'price': '가격(원가)',
24    'Final price': '할인 가격(Y)'
25 }, inplace=True)

```

✓ 0.0s
Python

	게임 ID	게임별 세일 순서	출시일수	세일시작 연도	세일시작 월	세일시작 일	세일종료 연도	세일종료 월	세일종료 일	세일 기간	...	최근 평가	최근 평가수	종합 평가	종합 평가수	게임 장르	총 판매금액	총 순이익	총 구매자수	가격(원가)	할인 가격(Y)	
	0	10	1	8495	2015	11	25	2015	12	1	7	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	5250
	1	10	2	8495	2015	12	22	2016	1	4	13	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	5250
	2	10	3	8495	2016	2	5	2016	2	12	7	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	2620
	3	10	4	8495	2016	11	23	2016	12	1	8	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	2620
	4	10	5	8495	2016	12	22	2017	1	2	12	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	2620
...	...	...	...	...	...	...	데이터를 정리하는 전처리는 끝났습니다. 이 데이터를 조정하고 변형하는 과정이 남았습니다.							...	...	...	...	...	...	...	...	
26010	985890	23	1375	2023	3	16								...	...	...	...	...				
26011	985890	24	1375	2023	6	29								...	...	...	...	...				
26012	985890	25	1375	2023	9	7								...	...	...	...	...				
26013	985890	26	1375	2023	11	21								...	...	...	...	...				
26014	985890	27	1375	2023	12	21								...	...	...	...	...				

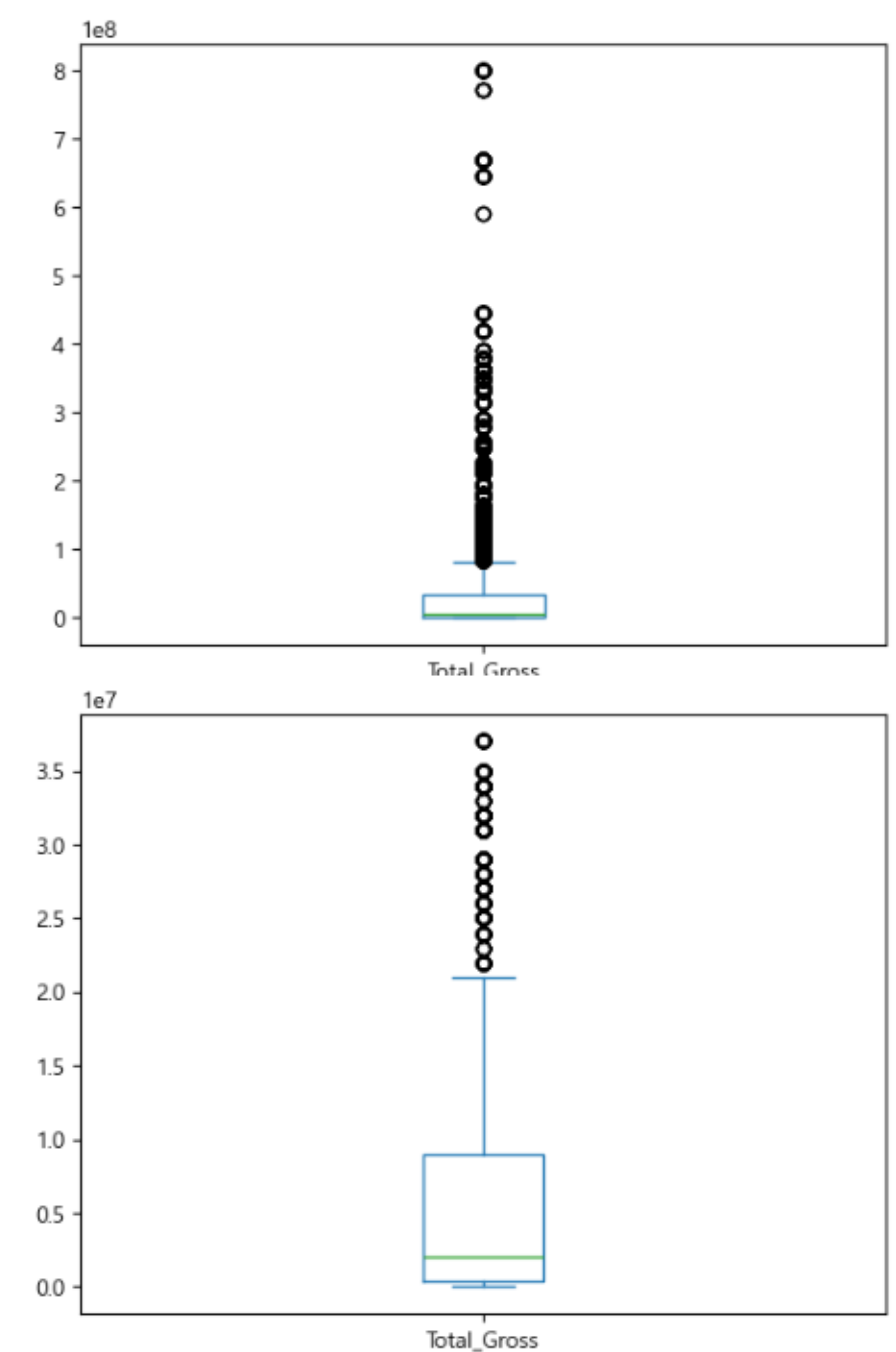
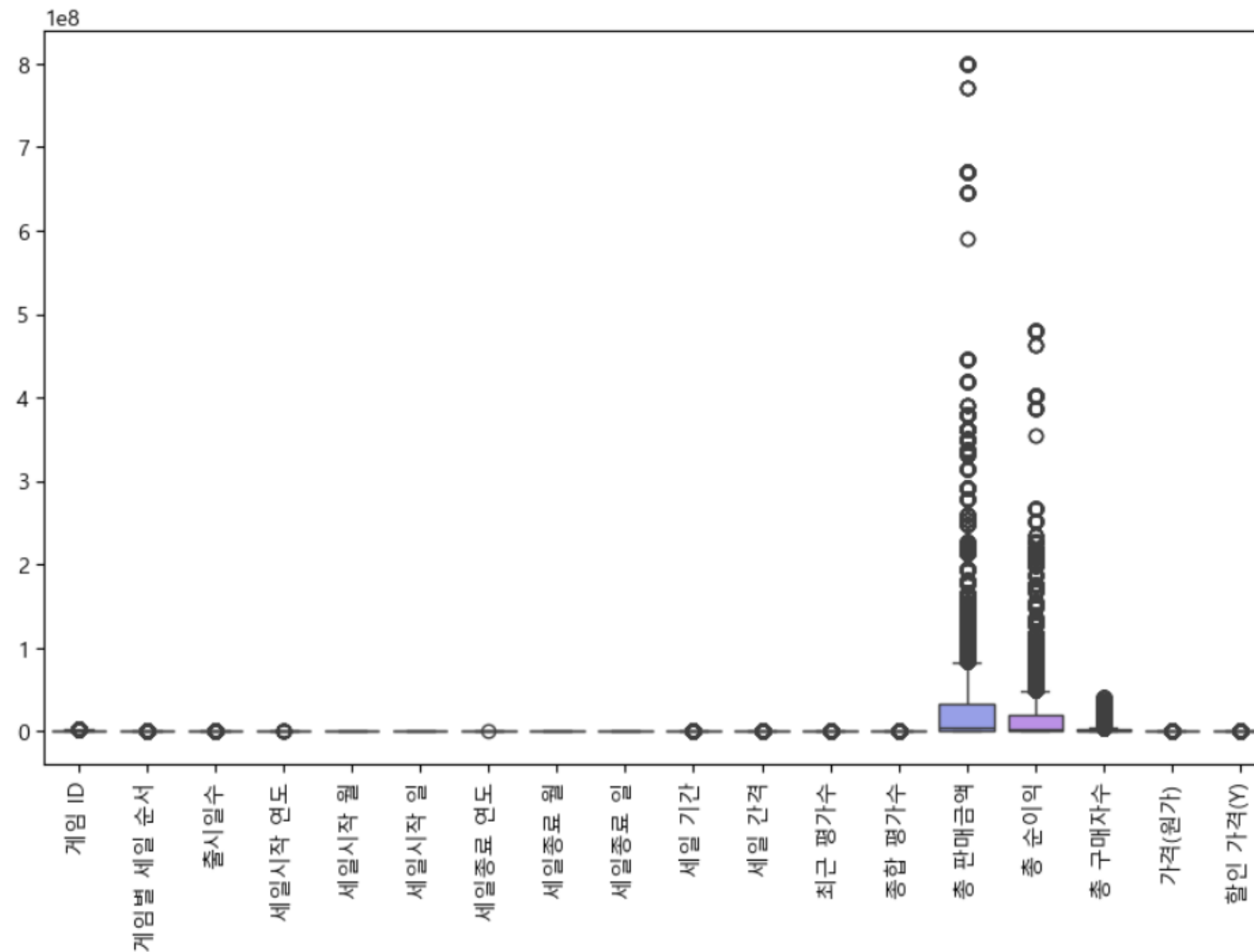
데이터를 정리하는 전처리는 끝났습니다.
이 데이터를 조정하고 변형하는 과정이 남았습니다.

26015 rows × 23 columns

EDA의 일환으로
박스 그래프를 그려줍니다.

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=new_order_df)
plt.xticks(rotation=90)
plt.show()
```

금액의 단위가 너무 커서
박스가 제대로 그려지지 않습니다.



이상치 제거

IQR(Interquartile Range)은
데이터의 중간 50% 범위를 측정하는 통계적 개념입니다.
IQR을 사용하여 이상치를 제거하는 것은
데이터의 중심적인 부분에 집중되는 경우에 유용합니다.

```
1 Q1 = np.percentile(new_order_df.Total_Gross, 25)
2 Q3 = np.percentile(new_order_df.Total_Gross, 75)
3 iqr = Q3 - Q1
4 upper_limit = Q3 + 1.5 * iqr
5 lower_limit = Q1 - 1.5 * iqr
6 drop_idx = new_order_df[(new_order_df.Total_Gross >= lower_limit) & (new_order_df.Total_Gross <= upper_limit)]
7
8 df = new_order_df
9 for col in df.columns[:-1]: # TARGET 컬럼을 제외하고 반복
10     iqr = np.percentile(df[col], 75) - np.percentile(df[col], 25) # 25~75%
11     upper_limit = np.percentile(df[col], 75) + iqr*100000 # 상한선 수치조정 1.5 -> 100000
12     lower_limit = np.percentile(df[col], 25) - iqr*100000 # 하한선 수치조정 1.5 -> 100000
13     drop_idx = df[col][((df[col] > upper_limit) | (df[col] < lower_limit)) & (df['Final price']!=0)].index
14     df.drop(list(drop_idx), inplace=True) # 범위를 벗어난 행을 삭제
```

get_dummies 함수

```
onehot_df = pd.get_dummies(new_order_df)
```

	게임 ID	게임별 세일 순서	출시 일수	세일 기간	세일 간격	최근 평 가수	종합 평 가수	총 판매금 액	총 순이익	총 구매자 수	...	게임 장르 _['Racing']	게임 장르_['Simulation', 'Early Access']	게임 장르_['Simulation', 'Sports', 'Strategy']	게임 장르 _['Simulation', 'Sports']	게임 장르_['Simulation', 'Strategy', 'Early Access']	게임 장르 _['Simulation', 'Strategy']	게임 장르 _['Simulation']	게임 장르 _['Strategy']	게임 장르 _['Utilities']	게임 장 르_[]
0	10	1	8495	7	21	1267	146475	50000000.0	30000000.0	9300000.0	...	False	False	False	False	False	False	False	False	False	False
1	10	2	8495	13	32	1267	146475	50000000.0	30000000.0	9300000.0	...	False	False	False	False	False	False	False	False	False	False
2	10	3	8495	7	285	1267	146475	50000000.0	30000000.0	9300000.0	...	False	False	False	False	False	False	False	False	False	False
3	10	4	8495	8	21	1267	146475	50000000.0	30000000.0	9300000.0	...	False	False	False	False	False	False	False	False	False	False
4	10	5	8495	12	171	1267	146475	50000000.0	30000000.0	9300000.0	...	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
26010	985890	23	1375	7	98	82	14585	6000000.0	4000000.0	300000.0	...	False	False	False	False	False	False	False	False	False	False
26011	985890	24	1375	14	55	82	14585	6000000.0	4000000.0	300000.0	...	False	False	False	False	False	False	False	False	False	False
26012	985890	25	1375	11	64	82	14585	6000000.0	4000000.0	300000.0	...	False	False	False	False	False	False	False	False	False	False
26013	985890	26	1375	8	22	82	14585	6000000.0	4000000.0	300000.0	...	False	False	False	False	False	False	False	False	False	False
26014	985890	27	1375	15	42	82	14585	6000000.0	4000000.0	300000.0	...	False	False	False	False	False	False	False	False	False	False

26015 rows × 1339 columns

컬럼 수가 23개에서 1339개로 증가

One-Hot Encoding 원-핫 인코딩

범주형 데이터를 모델이 이해할 수 있는 형태인
숫자형 데이터로 변환하는 기법입니다.

타겟 Y 설정

다중 회귀모델 : 여러개의 X 데이터를 넣어 Y를 예측하기

```
1 # 타겟 설정
2 y = onehot_df['할인 가격(Y)']
3 x = onehot_df.drop('할인 가격(Y)', axis=1)
```

예측하고자 하는 1개의 컬럼을 Y,
나머지 전체를 X로 설정합니다.

train_test_split 함수로 훈련/테스트 데이터 분리

x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2, random_state=111)

```
1 # 데이터를 shuffle 해서 비율에 맞춰 나누기
2 from sklearn.model_selection import train_test_split
3 x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2, random_state=111)
```

train_test_split 함수에서 test_size 매개변수를 사용하여 데이터를 분리할 때,
test_size에 할당된 비율만큼 데이터가 **테스트 세트**로 할당되고, 나머지 데이터는 **훈련 세트**로 할당됩니다.
(test_size=0.2로 설정하면 전체 데이터셋에서 20%가 테스트 세트로 할당되고, 나머지 80%가 훈련 세트로 할당됩니다.)

이때 데이터는 **랜덤**으로 섞여서 분리됩니다.
데이터를 무작위로 섞는 것은 이러한 편향을 방지하여
모델의 **일반화** 성능을 향상시키는 데 도움이 됩니다.

데이터 분리

데이터를 X, Y 각각의 **훈련 데이터**와 **검증 데이터**로 나누어
모델을 훈련하고 평가합니다.
일반적으로 70:30 또는 80:20의 비율로 나눕니다.

정규화는
데이터의 스케일을 조정하여
모든 특성이 **동등한 중요도**를
가지도록 만드는 과정입니다

이때, 트레인 데이터와
테스트 데이터를
모두 조정해야 합니다.

Normalization 정규화

SCALER 1

MinMaxScaler

최대값을 1, 최소값을 0으로 설정해
모든 값을 0~1사이의 값으로
압축해서 변형됩니다.

```
scaler = MinMaxScaler()
scaler.fit(x_train)
x_train_mm_scaled = scaler.transform(x_train)
x_test_mm_scaled = scaler.transform(x_test)
x_test_mm_scaled
```

```
array([[8.05803404e-02, 1.85185185e-02, 4.12314041e-01, ...,
        1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [3.30692809e-01, 2.96296296e-01, 1.48984689e-01, ...,
        1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [8.52103172e-02, 7.96296296e-01, 4.46954067e-01, ...,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       ...,
       [4.08202744e-01, 1.29629630e-01, 8.91519166e-02, ...,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [1.59267451e-01, 2.50000000e-01, 2.67564339e-01, ...,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [1.83699483e-04, 0.00000000e+00, 6.02345532e-01, ...,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00]])
```

SCALER 2

RobustScaler

로그를 취하면 큰 값이 뭉쳐지는 효과로
작은 값은 적게,
큰 값은 많이 변화됩니다.

```
scaler = RobustScaler()
scaler.fit(x_train)
x_train_rb_scaled = scaler.transform(x_train)
x_test_rb_scaled = scaler.transform(x_test)
x_train_rb_scaled
```

```
array([[ -0.31811509, -0.76923077,  0.83072454, ...,  0.
        0. , 0. ],
       [ 0.11563373,  0.26923077,  0.06777473, ...,  0.
        0. , 0. ],
       [ 0.47881344, -0.53846154, -0.56930773, ...,  0.
        0. , 0. ],
       ...,
       [ 1.6841878, -0.73076923, -1.03921252, ...,  0.
        0. , 0. ],
       [-0.34180604,  1.23076923,  0.53057931, ...,  0.
        0. , 0. ],
       [-0.24787703, -0.03846154,  0.57963531, ...,  0.
        0. , 0. ]])
```

SCALER 3

StandardScaler

+값 범위 내에서 정규화 시킵니다.
특성의 평균을 0,
std(표준편차)를 1로 만들도록
데이터를 변환합니다.

```
scaler = StandardScaler()
scaler.fit(x_train)
x_train_st_scaled = scaler.transform(x_train)
x_test_st_scaled = scaler.transform(x_test)
x_test_st_scaled
```

```
array([[ -0.96600359, -1.15974848,  1.02741666, ...,  5.10526386,
        -0.01834278, -0.06250754],
       [ 0.42971333,  0.42478599, -0.66425658, ...,  5.10526386,
        -0.01834278, -0.06250754],
       [-0.94016667,  3.27694804,  1.24995017, ..., -0.19587626,
        -0.01834278, -0.06250754],
       ...,
       [ 0.86224646, -0.5259347, -1.04863264, ..., -0.19587626,
        -0.01834278, -0.06250754],
       [-0.5269014,  0.16069691,  0.09751958, ..., -0.19587626,
        -0.01834278, -0.06250754],
       [-1.41464557, -1.26538412,  2.24821178, ..., -0.19587626,
        -0.01834278, -0.06250754]])
```

03.

머신러닝 학습 / 예측 / 평가



머신러닝 프로젝트
스팀 할인 가격 예측

STEAM SALE PRICE PREDICTION

01. 주제선정 및 데이터 수집

- 프로젝트 소개
- 스크롤링
- 셀레니엄

02. EDA / 전처리

- 탐색적 데이터 분석(Exploratory Data Analysis)
- 결측값 처리, 데이터 타입 변환, 특성(피쳐) 선택
- 이상치 제거, 원-핫 인코딩, 데이터 분리, 정규화(스케일링)

03. 머신러닝 학습 / 예측 / 평가

- 알고리즘 선택
- 알고리즘 학습 / 예측 / 평가 1
- 알고리즘 학습 / 예측 / 평가 2

04. 알고리즘 최적화 및 활용

- 하이퍼파라미터 튜닝 : 그리드 서치(Grid Search)
- 스팀 할인 가격 예측 알고리즘 결론
- 팀원 소개, 질의응답

회귀모델 Regression

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, ExtraTreesRegressor
from sklearn.tree import DecisionTreeRegressor
```

```
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from catboost import CatBoostRegressor
```

객체 생성

```
lr = LinearRegression()
rid = Ridge()
las = Lasso()
ele = ElasticNet()
```

```
rfr = RandomForestRegressor()
gbr = GradientBoostingRegressor()
etr = ExtraTreesRegressor()
```

```
dtr = DecisionTreeRegressor()
xgbr = XGBRegressor()
lgbmr = LGBMRegressor()
cbr = CatBoostRegressor()
```

회귀모델 평가지표 Model Evaluation

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

회귀모델
알고리즘 선택

할인 가격이 타겟 Y값

Classification 분류모델

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import VotingClassifier (위 4개 통합)
```

```
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier
```

```
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.naive_bayes import GaussianNB
from catboost import CatBoostClassifier
```

객체 생성

```
tree = DecisionTreeClassifier()
log = LogisticRegression()
svc = SVC(probability=True)
knn = KNeighborsClassifier()
```

```
bag = BaggingClassifier()
rfc = RandomForestClassifier()
ada = AdaBoostClassifier()
grad = GradientBoostingClassifier()
```

분류모델 평가지표

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve, roc_auc_score
```

```
xgb = XGBClassifier()
lgbm = LGBMClassifier()
nb = GaussianNB()
cat = CatBoostClassifier()
```

알고리즘 객체를 리스트로 묶어서
반복문으로 실행합니다.

```
1 # 데이터 학습, 예측, 평가 : 8개 모델, 최소결합
2 models = [lr, rid, las, ele, rfr, gbr, xgbr, lgbmr]
3 for m in models:
4     m.fit(x_train.values, y_train)
5     preds = m.predict(x_test.values)
6     score = r2_score(y_test, preds)
7     mse = mean_squared_error(y_test, preds)
8     mas = mean_absolute_error(y_test, preds)
9     print(m.__class__.__name__)
10    print(score)
11    print(mse)
12    print(mas)
13    print('-----')
```

Python

회귀모델 평가지표인

R2 score는 값이 높을수록,

MSE(mean squared error)와

MAE(mean absolute error)는

값이 낮을수록 더 예측을 잘 하는

알고리즘입니다.

1. 알고리즘 학습/예측/평가

할인가격 예측 알고리즘 1

LinearRegression
0.6479524391700485
39123293.57573348
3591.799445070204

Ridge
0.650088501948898
38885911.41351622
3593.7183343165316

Lasso
0.6470630548180774
39222131.48565054
3646.7716776081693

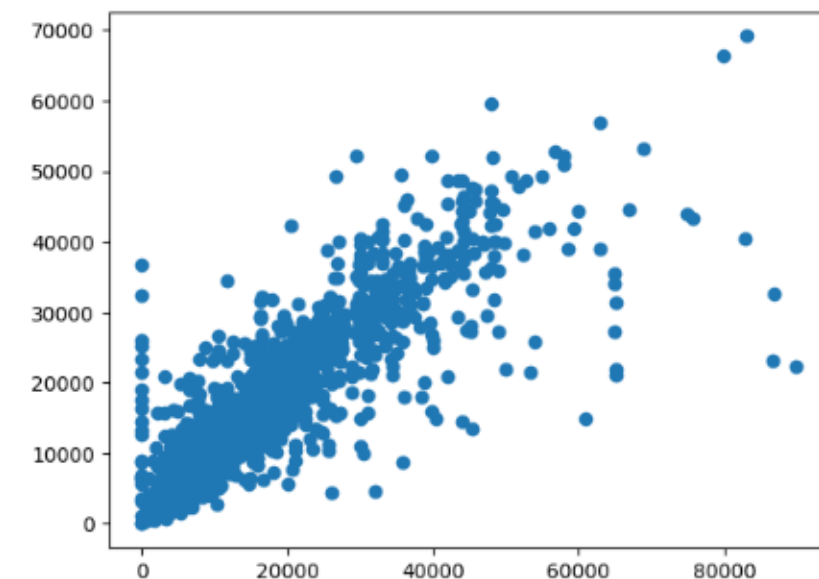
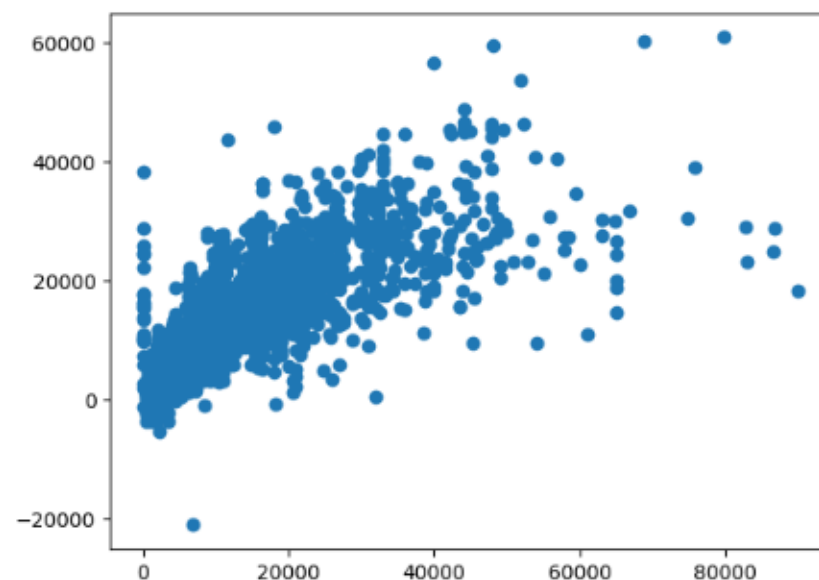
ElasticNet
0.5122354355565222
54205619.847426064
4794.160055986337

RandomForestRegressor
0.818648877286564
20153678.09247151
1974.2424431024567

GradientBoostingRegressor
0.6835387108802164
35168566.117601156
3615.3626353724294

XGBRegressor
0.7834628073507448
24063930.84533826
2537.3990103872297

LGBMRegressor
0.7915273263583196
23167715.163750764
2588.9558483784353



RandomForestRegressor가
0.81로 가장 좋게 나왔습니다.

LinearRegression 그래프보다
중앙으로 모이는 그래프를 보여줍니다.

할인가격 예측 알고리즘 2 : 모델, 날짜 추가

LinearRegression
0.6447851017351585
39475282.017370395
3643.8031359069123

Ridge
0.6464966604677602
39285075.2890243
3650.819522903947

Lasso
0.6439427376791174
39568894.528658494
3700.644474867248

ElasticNet
0.5125173735430629
54174287.88025782
4793.186076297073

RandomForestRegressor
0.8067244682312922
21478846.075702827
2024.297820932662

GradientBoostingRegressor
0.6865567502771032
34833169.272160254
3599.1842320863325

ExtraTreesRegressor
0.8016682398088574
22040748.3041703
1954.9327409415826

DecisionTreeRegressor
0.6711854856275161
36541388.74710377
2403.1838797140745

XGBRegressor
0.788359774598662
23519727.423450172
2521.8482561231335

LGBMRegressor
0.7911883282005553
23205388.267965946
2622.6118732593513

날짜 컬럼을 추가하자
예측값이 더 하락했습니다.

매출 데이터를 추가하고
알고리즘도 11개로 학습시켰습니다.

```
1 # 데이터 학습, 예측, 평가 (스케일링 없음)
2 models = [lr, rid, las, ele, rfr, gbr,
3            etr, dtr, xgbr, lgbmr, cbr]
4 for m in models:
5     m.fit(x_train.values, y_train)
6     preds = m.predict(x_test.values)
7     score = r2_score(y_test, preds)
8     mse = mean_squared_error(y_test, preds)
9     mas = mean_absolute_error(y_test, preds)
10    print(m.__class__.__name__)
11    print(score)
12    print(mse)
13    print(mas)
14    print('-----')
```

Python

RandomForestRegressor가
0.859로 지금까지
가장 좋은 수치가 나왔습니다.

3가지 정규화를 적용해 보았으나
스케일링을 적용하지 않았을 때가
조금 더 확률이 높았습니다.

2. 알고리즘 학습/예측/평가

할인가격 예측 알고리즘 3

LinearRegression
0.6844960872340218
34310387.72636099
3317.6297305511753

Ridge
0.7149418284848612
30999477.32346025
3302.657049667221

Lasso
0.7082954226539291
31722259.9953036
3393.0230150763214

ElasticNet
0.5757527784903504
46135994.11245419
4364.9368726538505

RandomForestRegressor
0.859598887265991
15268326.065703942
1607.9448548914088

GradientBoostingRegressor
0.7581210818838537
26303824.22405445
3151.8633895060602

ExtraTreesRegressor
0.8458644411441226
16761918.229163576
1617.9524639630986

DecisionTreeRegressor
0.7331841815715096
29015659.74745339
1927.6175283490295

XGBRegressor
0.8361460628739383
17818771.450406488
2244.87424860655

LGBMRegressor
0.8304349803557237
18439839.676848825
2392.6330265047386

CatBoostRegressor
0.8388003608810908
17530122.118078686
2320.397917720538

MinMaxScaler

LinearRegression
-1.0151225003716185e+21
1.1039243942225683e+29
6624663956439.817

Ridge
0.7145019817255231
31047309.734541215
3302.9889396189806

Lasso
0.7077362537266028
31783068.441490225
3393.5073463951903

ElasticNet
0.1352977127309134
94034557.23885007
6946.380733197296

RandomForestRegressor
0.8583247904526256
15406881.417630482
1619.3719815491063

GradientBoostingRegressor
0.7579719363684045
26320043.485545836
3152.5125651902576

ExtraTreesRegressor
0.8483494034371838
16491683.800015453
1610.5660465116277

DecisionTreeRegressor
0.7339155157479199
28936128.69210071
1941.839900057659

XGBRegressor
0.8361460628739383
17818771.450406488
2244.87424860655

LGBMRegressor
0.8304349803557237
18439839.676848825
2392.6330265047386

CatBoostRegressor
0.8388003608810908
17530122.118078686
2320.397917720538

RobustScaler

LinearRegression
-2.257489563586819e+17
2.4549724767543186e+25
107976642490.8582

Ridge
0.7149343733495623
31000288.053762805
3303.2996172138014

Lasso
0.7083223603688529
31719330.575394377
3392.600788966676

ElasticNet
0.5089549552811081
53400117.06260594
4832.39642406179

RandomForestRegressor
0.8590731642206964
15325497.342480224
1611.1734845281567

GradientBoostingRegressor
0.7579417488543
26323326.31428905
3153.253833193782

ExtraTreesRegressor
0.8449292915340496
16863613.7847669
1627.18665000961

DecisionTreeRegressor
0.735487177681911
28765213.75840861
1934.451854699212

XGBRegressor
0.8361460628739383
17818771.450406488
2244.87424860655

LGBMRegressor
0.8282664929486635
18675658.126971044
2398.5834934770655

CatBoostRegressor
0.8388003608810908
17530122.118078686
2320.397917720538

StandardScaler

LinearRegression
-5.163373482328424e+24
5.6150601937575206e+32
708247907616094.4

Ridge
0.7153188977137502
30958471.8369373
3292.345970266705

Lasso
0.7151301925998663
30978993.121654633
3293.2759650035855

ElasticNet
0.6738363360053701
35469613.27927156
3497.13507724032

RandomForestRegressor
0.858186922939804
15421874.220028099
1617.6387238131845

GradientBoostingRegressor
0.7581388611976929
26301890.761023883
3152.196936057706

ExtraTreesRegressor
0.84609833937421
16736482.287997426
1624.4337536036903

DecisionTreeRegressor
0.7447807370038254
27754558.70536229
1912.0960984047665

XGBRegressor
0.8361460628739383
17818771.450406488
2244.87424860655

LGBMRegressor
0.8301053376402181
18475687.629671387
2398.6425265725306

CatBoostRegressor
0.8388003608810908
17530122.118078686
2320.397917720538

04. 알고리즘 최적화 및 결론



머신러닝 프로젝트
스팀 할인 가격 예측

STEAM SALE PRICE PREDICTION

01. 주제선정 및 데이터 수집

- 프로젝트 소개
- 스크롤링
- 셀레니엄

02. EDA / 전처리

- 탐색적 데이터 분석(Exploratory Data Analysis)
- 결측값 처리, 데이터 타입 변환, 특성(피쳐) 선택
- 이상치 제거, 원-핫 인코딩, 데이터 분리, 정규화(스케일링)

03. 머신러닝 학습 / 예측 / 평가

- 알고리즘 선택
- 알고리즘 학습 / 예측 / 평가 1
- 알고리즘 학습 / 예측 / 평가 2

04. 알고리즘 최적화 및 결론

- 하이퍼파라미터 튜닝 : 그리드 서치(Grid Search)
- 스팀 할인 가격 예측 알고리즘 결론
- 팀원 소개, 질의응답

GridSearchCV

GridSearchCV는 주어진 하이퍼파라미터 그리드에서 모든 조합을 시도하고 교차 검증(Cross-validation)을 통해 각 조합의 성능을 평가하여 최적의 매개변수를 탐색하는 도구입니다. 모델의 일반화 성능을 최대화하는 최적의 하이퍼파라미터를 찾습니다.

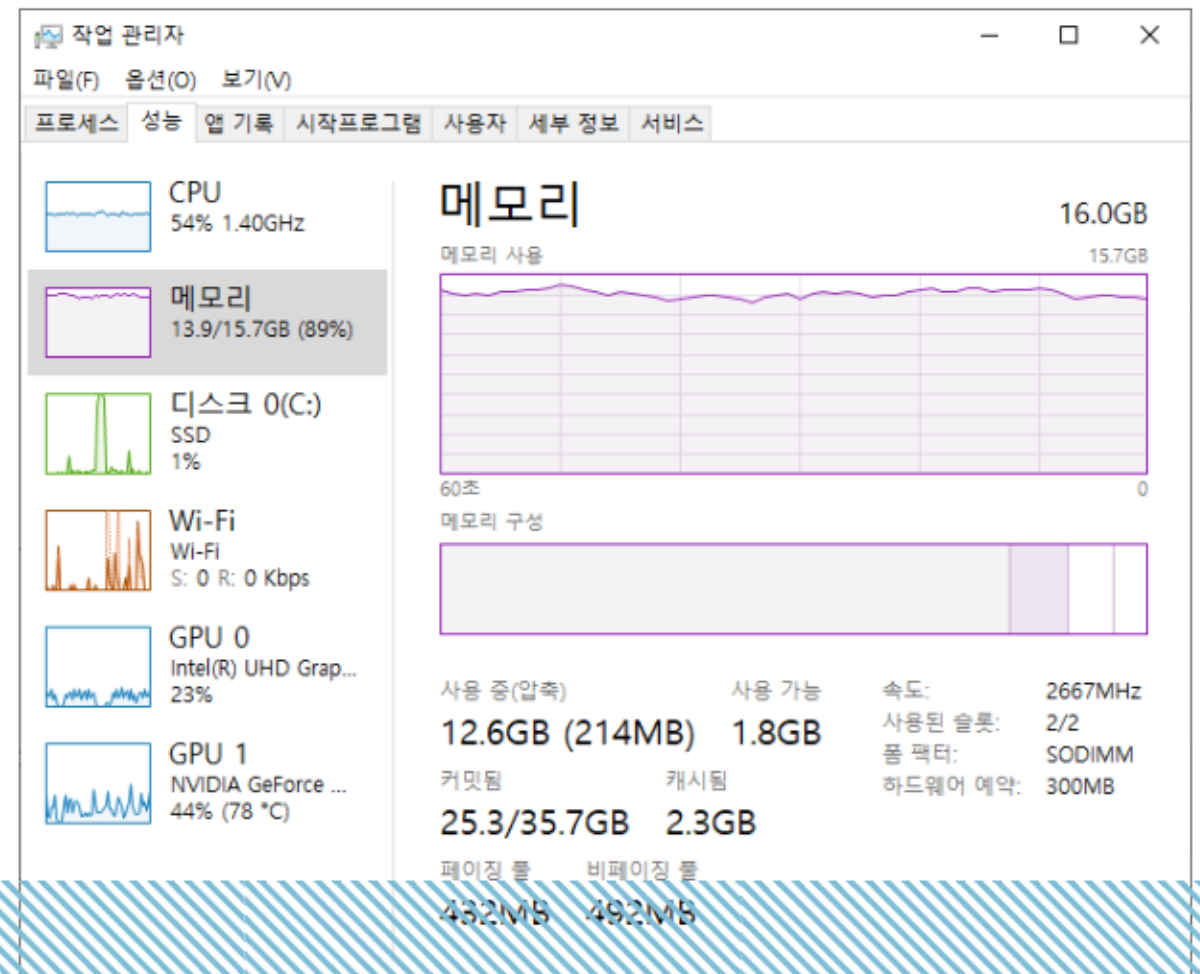
```
1 # GridSearchCV를 사용하여 최적의 조합을 찾음
2 rfr = RandomForestRegressor()
3
4 rfr_param_grid = {
5     'n_estimators': [50, 100, 150],      # 트리의 개수
6     'max_depth': [None, 10, 20, 30],     # 트리의 최대 깊이
7     'min_samples_split': [2, 5, 10],     # 노드를 분할하기 위한 최소한의 샘플 개수
8     'min_samples_leaf': [1, 2, 4],       # 리프 노드가 되기 위한 최소한의 샘플 개수
9     'max_features': ['auto', 'sqrt', 'log2'], # 각 노드에서 분할에 사용할 특성의 최대 개수
10     'random_state': [42]                # 랜덤 시드
11 }
```

```
1 grid_search = GridSearchCV(estimator=rfr, param_grid=rfr_param_grid, cv=5, n_jobs=-1, verbose=2)
2 grid_search.fit(x_train.values, y_train)
```

최적의 파라미터 값 출력
print("Best parameters:", grid_search.best_params_)
Best parameters: {'max_depth': **None**, 'max_features': **sqrt**, 'min_samples_leaf': **1**, 'min_samples_split': **2**, 'n_estimators': **150**, 'random_state': **42**}

```
GridSearchCV(cv=5, estimator=RandomForestRegressor(), n_jobs=-1,
param_grid={'max_depth': [None, 10, 20, 30],
'max_features': ['auto', 'sqrt', 'log2'],
'min_samples_leaf': [1, 2, 4],
'min_samples_split': [2, 5, 10],
'n_estimators': [50, 100, 150], 'random_state': [42]},
verbose=2)
* estimator: RandomForestRegressor
RandomForestRegressor()
* RandomForestRegressor
RandomForestRegressor()
```

RandomForestRegressor가 0.818에서 **0.859**로 지금까지 가장 좋은 수치가 나왔기 때문에 GridSearch를 적용해 봅니다. 16기가 메모리 90%로 약 2시간이 소요됩니다.



Grid Search 하이퍼파라미터 튜닝

```
1 best_params = {
2     'max_depth': None,
3     'max_features': 'sqrt',
4     'min_samples_leaf': 1,
5     'min_samples_split': 2,
6     'n_estimators': 150,
7     'random_state': 42
8 }
```

하지만 추출된 파라미터값을 적용해보면 오히려 예측확률이 더 떨어집니다.
R2 Score: **0.819**94399452972
Mean Squared Error: 19580712.346751127
Mean Absolute Error: 2176.959612189549

스팀 할인 가격 예측 알고리즘 : 시연



2015년 올해의 게임

출시 6주만에 6백만 장 판매 6330만 달러의 순이익(한화로 약 755억)
콘솔까지 총 누적 판매량 4천만 장 추산

테스트 소스

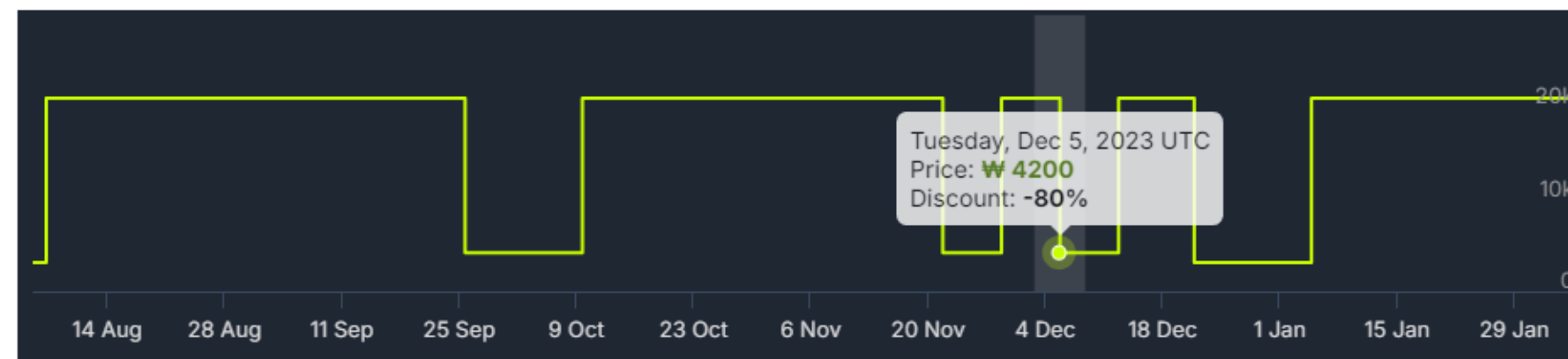
```
1 last_count_index = onehot_df.reset_index().sort_values('게임별 세일 순서').groupby(
2 ('게임 ID')[['할인 가격(Y)', 'index']].last()['index'].unique()
3 find_price_df = onehot_df.iloc[last_count_index]
4 x_find_test = find_price_df.drop(['할인 가격(Y)'], axis=1)
5 y_find_test = find_price_df['할인 가격(Y)']
6
7 id = 20920 # 조회할 게임 id : 위쳐 3
8
9 preds_price = rfr.predict(x_find_test[x_find_test['게임 ID'] == id])
10
11 onehot_df[onehot_df['게임 ID'] == id][-1:].index # 게임 가격 번호
12
13 # 검색한 가격 확인
14 confirm_price = y_find_test[x_find_test[x_find_test['게임 ID'] == id].index].values[0]
15
16 print('확인가격 : ', confirm_price)
17 print('예측가격 : ', preds_price[0])
```

✓ 0.1s

Python

게임아이디를 입력한 한 개 게임의 최종 할인데이터 1개만 분리

직전 할인 가격 : 4,200원



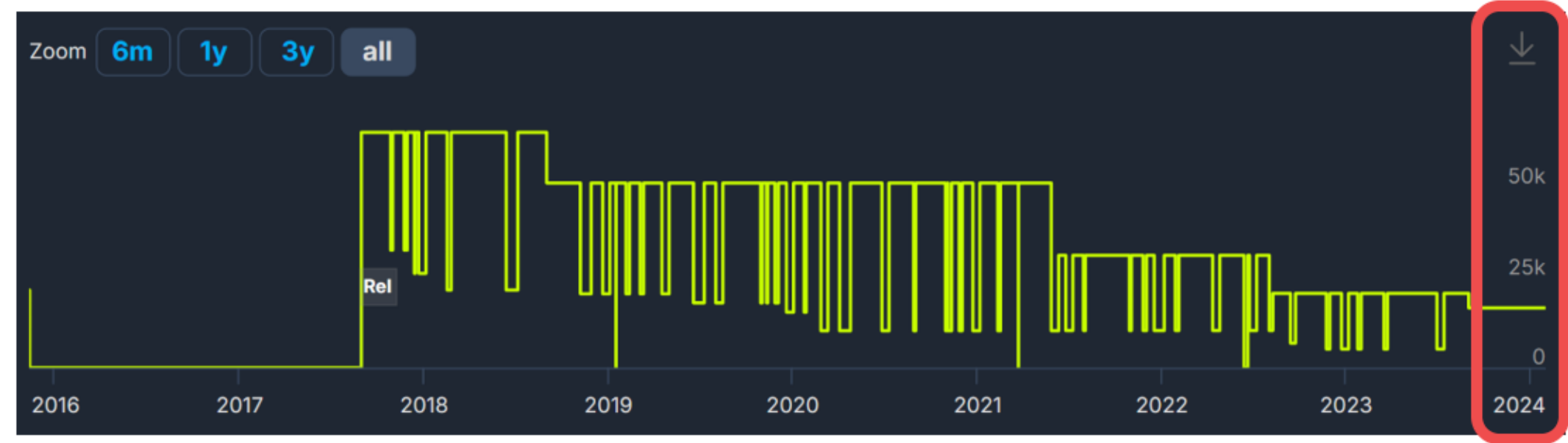
실제 할인 가격 : 3,150원 (정답)



예측가격 : 3,444원

가격 하락과 인근 값을 예측하였습니다. 실제 세일은 % 단위로 이루어지기 때문에 인근 값이 큰 의미가 있습니다.

스팀 할인 가격 예측 서비스



When is the next major Steam sale?

Spring Sale 2024

on 14 March 2024

Your local time: 15 March 2024 at 02:00

38 : 21 : 05 : 23

	게임 ID	게임별 세일 순서	출시일수	세일시작 연도	세일시작 월	세일시작 일	세일종료 연도	세일종료 월	세일종료 일	세일 기간	...	최근 평가	최근 평가수	종합 평가	종합 평가수	게임 장르	총 판매금액	총 순이익	총 구매자수	가격(원가)	할인 가격(Y)
0	10	1	8495	2015	11	25	2015	12	1	7	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	5250
1	10	2	8495	2015	12	22	2016	1	4	13	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	5250
2	10	3	8495	2016	2	5	2016	2	12	7	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	2620
3	10	4	8495	2016	11	23	2016	12	1	8	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	2620
4	10	5	8495	2016	12	22	2017	1	2	12	...	Overwhelmingly Positive	1267	Overwhelmingly Positive	146475	['Action']	50000000.0	30000000.0	9300000.0	10500	2620

스팀 게임의 확정된 할인률을 정리해서 소개하는 곳은 많지만, 그 다음 할인을 예측하는 서비스는 없습니다.
같은 DB를 이용하지만 머신러닝을 통해 차별화된 게임 구매 정보를 제공할 수 있습니다

85.9% 확률로 예측

결론

스팀 게임의 다음 할인가격을 예측하는 데에는
RandomForestRegressor 알고리즘이 85.9%로 가장 좋은 성능을 보였습니다.
피쳐 조정, 그리드 서치 등 예측 확률 개선의 여지가 더 있는 **경우의 수**를 지속적으로 시도해 보는
과정에서 머신러닝의 프로세스를 충실히 배울 수 있었습니다.
다음에는 이러한 시계열 데이터를 딥 러닝을 통해 적용하고
홈페이지를 만들어 실제 서비스로도 운영해보고 싶습니다.

2조

팀장
이도규



데이터를 수집, 정제, 전처리를 하는 일련의 과정에서 신경 써야 할 요소가 상당히 많다는 것을 알게 되었습니다. 이런 어려움을 통해 목표값에 가까워지는 즐거움을 느낄 수 있었습니다. 감사합니다.

팀원
윤연석



예제로만 공부하다가 실제 자료를 수집하며 프로젝트 전체를 진행해 보니 스크롤링 단계의 피겨 선정과 전처리부터, 최종 하이퍼파라미터 튜닝까지 알고리즘 최적화를 위해 각 단계별로 신경 쓸 것이 훨씬 더 많다는 것을 절감했습니다. 모두 한데 힘을 모아서 좋은 결실을 본 것 같아 뿌듯합니다.

팀원
강진성



이번 프로젝트를 통해서 수업 시간에 배운 머신러닝을 더 깊이 공부하고 적용해 볼 수 있는 뜻깊은 시간이 되었습니다. 각자 맡은 부분을 하시느라 팀원분 모두들 고생하셨습니다.

팀원
이동준



실제 모듈학습을 진행했을 때 각 모듈들의 성능과 데이터별로 최적의 모델들이 다른 것들을 보고 같은 데이터라도 어떤 가공을 거치느냐에 따라 더 좋은 모델이 될 수도 있고 나쁜 모델이 될 수도 있다는 것을 깨달았습니다. 향후에는 다양한 학습 방식을 통해 좀 더 효율이 좋은 모델을 만들고 싶습니다.